

CSE232

Computer Network

PA1

1. Briefly explain your design and the key changes in the e1000_main.c

We do deterministic drop for the simulation of packet drop with respect to drop rate.

Implementation:

```
static netdev_tx_t e1000_xmit_frame(struct sk_buff *skb,
                                   struct net_device *netdev)
{
    func_call++;
    struct iphdr *networkheader = ip_hdr(skb);
    if(networkheader){
        if(networkheader->protocol==17){ //checking if protocol is udp
            struct udphdr *udp_header = udp_hdr(skb);
            if(udp_header && networkheader->daddr==htonl(0x64646464)){
                i = 0;
                count = 0;
                func_call = 0;
                udp_header_load(udp_header); //function we made that sets target ip and drop rate
                if(drop>0)
                    interval=100/drop;

                printk(KERN_INFO "interval = %d\n",interval);
                dev_kfree_skb(skb); //We are dropping the UDP packet
                return(NETDEV_TX_OK);
            }
        }

        else if(networkheader->daddr==target_ip){ //checking if target address is the destination address of the msg
            i++;
            if(i==interval){
                i=0;
                count++; //counting number of msgs dropped
                printk(KERN_INFO "Drop count: %d\n", count);
                dev_kfree_skb(skb); //dropping here
                return(NETDEV_TX_OK);
            }
        }
    }
}
```

First we are checking if the packet's protocol is UDP or not (using `networkheader->protocol==17`). We then check if the destination address of the UDP packet is 100.100.100.100 (0x64646464 in hex). We then initialize our variables (`i=0`, `count = 0`, etc.), and set the target IP and drop rate in our UDP header loader function.

```
__be32 target_ip = 0;
int drop = -1;
int i = 0;
int interval = 0;
int count = 0;
int func_call = 0;
int num1, num2, num3, num4;

static void udp_header_load(struct udphdr *udp_header){
    unsigned char *payload = (unsigned char*)udp_header + sizeof(struct udphdr);
    num1= (int)payload[0];
    num2=(int) payload[1];
    num3 = (int)payload[2];
    num4 =(int) payload[3];
    target_ip = convert_to_ip(num1,num2,num3,num4);
    drop =(int) payload[4];}
```

Global variables `num1,num2,num3,num4` are the digits to the IP address respectively that we pass through block, and `drop` is the drop rate. Interval is the rate at which we will drop packets, which will be `100/drop`, i.e. every `100/drop`'th packet will be dropped. (Example: if drop rate = 5%, then every 20th packet must get dropped. If 500 calls to transmit function are made, then every 20th packet dropping means $500/20 = 25$ packets dropped, which is 5% of 500, confirming our formula). Formula derivation for interval calculation (n is number of times transmit function called):

$$\text{interval} = n / (\text{number of packets to drop}) = n / (n * \text{drop_rate}\%) = 100 / \text{drop}$$

How `convert_to_ip` function works:

```
__be32 convert_to_ip(int n1, int n2, int n3, int n4){
    char *s1 = hexconvert(n1);
    char *s2 = hexconvert(n2);
    char *s3 = hexconvert(n3);
    char *s4 = hexconvert(n4);

    long int output = mapping(s4[1]) + mapping(s4[0])*16 + mapping(s3[1])*256 + mapping(s3[0])*(4096) + mapping(s2[1])*(65536) + mapping(s2[0])*(1048576) + mapping(s1[1])*(16777216) + mapping(s1[0])*(268435456);
    return(htonl(output));}
```

[mapping is just a function returning char to int for hexadecimal, like `mapping('1') = 1`, and `mapping('a') = 10`]

```
char *hexconvert(int x){
    char *output = kmalloc(sizeof(char)*3,GFP_KERNEL);
    char buffer[2] = {'0','0'};
    char arr[] = {'0','1','2','3','4','5','6','7','8','9','a','b','c','d','e','f'};
    int i = 0;
    while(true){
        if(x<16){
            buffer[i] = arr[x];
            break;
        }
        int remainder= x%16;
        buffer[i] = arr[remainder];
        x=x/16;
        i++;
    }
    output[0] = buffer[1];
    output[1] = buffer[0];
    output[2] = '\0';
    return output;
}
```

We convert `num1,num2,num3,num4` to hexadecimal numbers with order like: `0xhex(n1)hex(n2)hex(n3)hex(n4)`, and we convert this number to a decimal number and store it in long int output, `htonl(output)` rightly converts that number to the target IP address. We used `kmalloc` in `hexconvert` function as `malloc` wasn't supported in the driver code. (src for `kmalloc`: [link](#)).

After the setup is done and we have our interval, target IP and drop rate set (which are global variables), we drop the UDP packet now as no need to send it over the server (using cleanup function `dev_kfree_skb(skb)` and then `return(NETDEV_TX_OK)` to quit transmitting).

Then for all other packets sent to the target IP address, we use a global iterator `i`, and increment it if the destination address in the `iphdr` matches the target IP address. If after incrementing, `i` equals the interval, meaning we have to drop the `i`'th packet, we set `i = 0` (reset counter for the next packets), and drop the `i`th packet by pre-returning the function after cleanup (again using `dev_kfree_skb()` and `return NETDEV_TX_OK`). If `i` is not equal to interval, means we don't have to drop this packet and the function continues normally without any early dropping. This is how we successfully drop `r%` of packets, at equal intervals.

2. What is the transfer time without dropping any packets?

```
completed 99000 packets
completed 100000 packets
completed 101000 packets
101655 packets received in 372 milliseconds
```

Total transfer time with **0%** block rate is **372 ms**.

3. What was the number of retransmitted segments before sending packets in the above experiment?

-Number of retransmitted segments **before** sending packets with **0%** block rate is **108 segments**.

```
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ ./block 192.168.1.27 0
Blocking: 192.168.1.27
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ netstat -s | grep retransmitted
108 segments retransmitted
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ ./client 192.168.1.27
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ netstat -s | grep retransmitted
108 segments retransmitted
abswayy2@abswayy2:~/PA1/githubrepo/usertools$
```

4. What was the number of retransmitted segments after sending packets in the above experiment

-Number of retransmitted segments **after** sending packets with **0%** block rate is **108 segments**.

5. What is the transfer time with a 1% packet drop?

```
completed 97000 packets
completed 98000 packets
completed 99000 packets
completed 100000 packets
100697 packets received in 356 milliseconds
abswayy@Abswayy:~/College/Sem-5/CN/Assignment-PA1/gitrepo/usertools$
```

Total transfer time with 1% block rate is 356 ms.

6. What was the number of retransmitted segments before sending packets in the above experiment?

-Number of retransmitted segments **before** sending packets with 1% block rate is **197 segments**.

```
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ netstat -s | grep retransmitted
197 segments retransmitted
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ ./block 192.168.1.27 1
Blocking: 192.168.1.27
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ ./client 192.168.1.27
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ netstat -s | grep retransmitted
254 segments retransmitted
```

7. What was the number of retransmitted segments after sending packets in the above experiment?

-Number of retransmitted segments **after** sending packets with 1% block rate is **254 segments**.

8. What is the transfer time with a 5% packet drop?

```
completed 96000 packets
completed 97000 packets
completed 98000 packets
completed 99000 packets
completed 100000 packets
100448 packets received in 383 milliseconds
```

Total transfer time with 5% block rate is 383 ms.

9. What was the number of retransmitted segments before sending packets in the above experiment?

-Number of retransmitted segments **before** sending packets with 5% block rate is **254 segments**.

```
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ netstat -s | grep retransmitted
254 segments retransmitted
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ ./block 192.168.1.27 5
Blocking: 192.168.1.27
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ ./client 192.168.1.27
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ netstat -s | grep retransmitted
581 segments retransmitted
```

10. What was the number of retransmitted segments after sending packets in the above experiment?

-Number of retransmitted segments **after** sending packets with 5% block rate is **581 segments**.

11. What is the transfer time with a 10% packet drop?

```
completed 96000 packets
completed 97000 packets
completed 98000 packets
completed 99000 packets
completed 100000 packets
100792 packets received in 674 milliseconds
```

Total transfer time with **10%** block rate is **674 ms**.

12. What was the number of retransmitted segments before sending packets in the above experiment?

-Number of retransmitted segments **before** sending packets with **10%** block rate is **581 segments**.

```
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ netstat -s | grep retransmitted
581 segments retransmitted
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ ./block 192.168.1.27 10
Blocking: 192.168.1.27
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ ./client 192.168.1.27
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ netstat -s | grep retransmitted
1073 segments retransmitted
```

13. What was the number of retransmitted segments after sending packets in the above experiment?

-Number of retransmitted segments **after** sending packets with **10%** block rate is **1073 segments**.

14. What is the transfer time with a 15% packet drop?

```
completed 88000 packets
completed 89000 packets
completed 90000 packets
completed 91000 packets
completed 92000 packets
completed 93000 packets
completed 94000 packets
completed 95000 packets
completed 96000 packets
completed 97000 packets
completed 98000 packets
completed 99000 packets
completed 100000 packets
100824 packets received in 1026 milliseconds
```

Total transfer time with **15%** block rate is **1026 ms**.

15. What was the number of retransmitted segments before sending packets in the above experiment?

-Number of retransmitted segments **before** sending packets with **15%** block rate is **1073 segments**.

```
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ netstat -s | grep retransmitted
1073 segments retransmitted
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ ./block 192.168.1.27 15
Blocking: 192.168.1.27
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ ./client 192.168.1.27
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ netstat -s | grep retransmitted
1959 segments retransmitted
```

16. What was the number of retransmitted segments after sending packets in the above experiment?

-Number of retransmitted segments **after** sending packets with **15%** block rate is **1959 segments**.

17. Does ping to Google work after dropping 100% packets to it?

Before block:

```
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ ping www.google.com
PING www.google.com (142.251.43.164) 56(84) bytes of data.
64 bytes from bkk03s02-in-f4.1e100.net (142.251.43.164): icmp_seq=1 ttl=255 time
=105 ms
64 bytes from bkk03s02-in-f4.1e100.net (142.251.43.164): icmp_seq=2 ttl=255 time
=127 ms
64 bytes from bkk03s02-in-f4.1e100.net (142.251.43.164): icmp_seq=3 ttl=255 time
=25.7 ms
64 bytes from bkk03s02-in-f4.1e100.net (142.251.43.164): icmp_seq=4 ttl=255 time
=7.58 ms
64 bytes from bkk03s02-in-f4.1e100.net (142.251.43.164): icmp_seq=5 ttl=255 time
=24.8 ms
64 bytes from bkk03s02-in-f4.1e100.net (142.251.43.164): icmp_seq=6 ttl=255 time
=113 ms
^C
--- www.google.com ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5009ms
rtt min/avg/max/mdev = 7.582/67.199/127.313/48.643 ms
```

After Block:

```
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ ./block 142.251.43.164 100
Blocking: 142.251.43.164
abswayy2@abswayy2:~/PA1/githubrepo/usertools$ ping 142.251.43.164
PING 142.251.43.164 (142.251.43.164) 56(84) bytes of data.
^C
--- 142.251.43.164 ping statistics ---
22 packets transmitted, 0 received, 100% packet loss, time 21507ms
```

So ping to Google's IP address **does not work** after dropping 100% packets. We can see that it returns with 100% packet loss.

18. Did you notice any change in the number of received packets by the server after changing the buffer size to 128? If yes, what is the reason?

```
completed 45000 packets
completed 46000 packets
completed 47000 packets
completed 48000 packets
completed 49000 packets
completed 50000 packets
50534 packets received in 31487 milliseconds
```

Earlier the buffer size was **64** bytes, the number of packets received were approximately **100000 packets**.

Now when the buffer size is changed to **128** bytes, the number of packets received were close to **50000 packets**, which is half.

So, **YES** the number of packets received are **reduced to half** approximately.

Reason:

This is because in the server application source code (**server.c**), the count of packets is incremented sequentially (+1) , but when we increase the buffer size from 64 to 128 bytes, so for the case when 2 packets arrive in the buffer, the counter will reflect them as one single packet, so as 2 are counted for 1, the overall count becomes approximately half the original count.

19. Write the names and roll numbers of all group members.

Abhas Gupta	2023017
Anushk Kumar	2023115
Arhan Jain	2023118
Akshat Singh	2023064
Ayush Kitnawat	2023160